

Contents

1 Data Structures

1.1	Disjoint Set Union (DSU)	1
1.2	Dynamic Fenwick Tree2d	1
1.3	Fenwick Tree	1
1.4	Fenwick Tree2d	2
1.5	Dual Segment Tree	2
1.6	Dynamic Segment Tree	2
1.7	Lazy Segment Tree	3
1.8	Persistent Segment Tree	3
1.9	Segment Tree	3
1.10	Disjoint Sparse Table	4
1.11	Sparse Table	4

2 Graphs

2.1	Lowest Common Ancestor (LCA)	4
2.2	Hld Commutative Lazy	5
2.3	Hld Commutative	5
2.4	Hld Non Commutative Lazy	5
2.5	Hld Non Commutative	6

3 Extra

3.1	Hash Function	6
-----	---------------	---

1 Data Structures

1.1 Disjoint Set Union (DSU)

```

/* DSU (Disjoint Set Union / Union-Find)
 * Mantem conjuntos disjuntos com uniao e busca eficiente.
 * Complexidade:  $O(a(N))$  por operacao, onde a e a inversa de Ackermann.
 * Memoria:  $O(N)$ 
 * Metodos:
 * - find(i): Retorna o representante do conjunto de i.
 * - join(i, j): Une os conjuntos de i e j, retorna true se uniu.
 * - same(i, j): Verifica se i e j estao no mesmo conjunto.
 * - size(i): Retorna o tamanho do conjunto de i.
 */

```

```

DS6 struct DSU {
1A8     int n;
923     vector<int> parent;
1DF     vector<int> sz;
75E     int num_sets;

DDB     DSU(int _n : n(_n), parent(_n), sz(_n, 1), num_sets(_n) {
4D6         iota(parent.begin(), parent.end(), 0);
548     }

C42     int find(int i) {
331         if (parent[i] == i) return i;
565         return parent[i] = find(parent[i]);
72C     }

D58     bool join(int i, int j) {
477         i = find(i), j = find(j);
41E         if (i != j) {
4C2             if (sz[i] < sz[j]) swap(i, j);
1F0             parent[j] = i;
529             sz[i] += sz[j];
CA8             num_sets--;
8A6             return true;
20F         }
D1F         return false;
30C     }

```

```

00C     int size(int i) { return sz[find(i)]; }
DA3     bool same(int i, int j) { return find(i) == find(j); }
FF6 };

```

1.2 Dynamic Fenwick Tree2d

```

/* Fenwick Tree 2D Sparse (Point Update, Rectangle Query)
 * Suporta coordenadas ate  $10^9$  usando compressao interna.
 * Complexidade: Build  $O(P \log P)$ , Update/Query  $O(\log^2 P)$ .
 * Memoria:  $O(P \log P)$ .
 * Requisitos:
 * - NODE deve ter: operator+=, operator-, operator+ e construtor default.
 * - OffLine: Todos os pontos de interesse devem ser passados no build.
 *
 * Creditos: Adaptado de TFG (Thiago Ferreira Goncalves)
 * Fonte: github.com/tfg58/Competitive-Programming
 */

/* --- Exemplo de NODE (Soma com Inclusao-Exclusao) ---
struct Node {
    ll val;
    Node(Long long v = 0) : val(v) {}

    void operator+=(const Node& other) {
        val += other.val;
    }

    Node operator-(const Node& other) const {
        return Node(val - other.val);
    }

    Node operator+(const Node& other) const {
        return Node(val + other.val);
    }
};
*/

```

```

8A0 template<typename NODE>
222 struct FenwickTree2DSparse {
6C1     vector<int> ord;
803     vector<vector<NODE>> bit;
E76     vector<vector<int>> coord;

BEF     FenwickTree2DSparse(vector<pii> pts) {
CD7         sort(pts.begin(), pts.end());
5A8         for (auto [x, y] : pts) {
DC4             if (ord.empty() || x != ord.back()) ord.push_back(x);
ABA         }

261         bit.resize(ord.size() + 1);
3EB         coord.resize(ord.size() + 1);

A0A         sort(pts.begin(), pts.end(), [](const pii& a, const pii& b) {
463             return a.second < b.second;
19C         });

5A8         for (auto [x, y] : pts) {
A1A             for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
3E1                 if (coord[i].empty() || coord[i].back() != y)
739                     coord[i].push_back(y);
D32             }
BAF         }

BE8         for (int i = 0; i < (int)bit.size(); i++) {
F3E             bit[i].assign(coord[i].size() + 1, NODE());
B33         }
84E     }

F1C     inline int get_x(int x) {
2C8         return upper_bound(ord.begin(), ord.end(), x) - ord.begin();
FF7     }

```

```

DD7     inline int get_y(int i, int y) {
190         return upper_bound(coord[i].begin(), coord[i].end(), y)
7A2             - coord[i].begin();
B44     }

5CF     void update(int x, int y, NODE v) {
A1A         for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
511             for (int j = get_y(i, y); j < (int)bit[i].size(); j += j & -j) {
9E0                 bit[i][j] += v;
66B             }
638         }
0CE     }

84C     NODE query(int x, int y) {
ADA         NODE res;
FAD         for (int i = get_x(x); i > 0; i -= i & -i) {
263             for (int j = get_y(i, y); j > 0; j -= j & -j) {
01D                 res += bit[i][j];
359             }
92F         }
B50         return res;
7C4     }

FCA     NODE query(int x1, int y1, int x2, int y2) {
CD6         return query(x2, y2) - query(x1 - 1, y2)
CAD             - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
944     }
67D };

```

1.3 Fenwick Tree

```

/* Fenwick Tree (Point Update, Prefix Query)
 * Realiza atualizacoes pontuais e consultas de prefixo.
 * Complexidade:  $O(\log N)$  para update e query.
 * Memoria:  $O(N)$ 
 * Requisitos:
 * - NODE deve ter: operator+=, operator- (para range query) e
 *   construtor default.
 */

```

```

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

```

```

8A0 template<typename NODE>
0F1 struct FenwickTree {
060     int N;
9B8     vector<NODE> bit;

5D2     FenwickTree(int n) : N(n), bit(n + 1) {}

D6B     void update(int idx, NODE v) {
B2F         for (idx++; idx <= N; idx += idx & -idx) {
046             bit[idx] += v;
23E         }
9DA     }

FCA     NODE query(int idx) { // [0, idx]
ADA         NODE res;
159         for (idx++; idx > 0; idx -= idx & -idx) {
8E6             res += bit[idx];
A38         }
B50         return res;
C00     }

762     NODE query(int l, int r) { // [l, r] (precisa ser reversivel)

```

```

7C3         if (l > r) return NODE();
A4E         return query(r) - query(l - 1);
821     }
134 };

```

1.4 Fenwick Tree2d

```

/* Fenwick Tree 2D (Point Update, Rectangle Query)
 * Realiza atualizacoes pontuais e consultas de prefixo em matrizes.
 * Complexidade: O(Log N * log M) para update e query.
 * Memoria: O(N * M)
 * Requisitos:
 * - NODE deve ter: operator+=, operator- e construtor default.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

8A0 template<typename NODE>
2E6 struct FenwickTree2D {
610     int N, M;
803     vector<vector<NODE>> bit;

237     FenwickTree2D(int n, int m) : N(n), M(m), bit(n + 1, vector<NODE>(m + 1)) {}

274     void update(int r, int c, NODE v) { // point update
3BA         for (int i = r + 1; i <= N; i += i & -i) {
786             for (int j = c + 1; j <= M; j += j & -j) {
9ED                 bit[i][j] += v;
95F             }
A1F         }
BF9     }

82E     NODE query(int r, int c) { // [(0,0), (r,c)]
ADA         NODE res;
8D9         for (int i = r + 1; i > 0; i -= i & -i) {
83F             for (int j = c + 1; j > 0; j -= j & -j) {
01D                 res += bit[i][j];
CB5             }
E20         }
B50         return res;
943     }

05F     NODE query(int r1, int c1, int r2, int c2) { // [(r1,c1), (r2,c2)]
5C4         if (r1 > r2 || c1 > c2) return NODE();
B7F         return query(r2, c2) - query(r1 - 1, c2)
794             - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
0E7     }
FF7 };

```

1.5 Dual Segment Tree

```

/* Dual Segment Tree (Range Update, Point Query)
 * Realiza atualizacoes em range e consultas pontuais.
 * Complexidade: O(Log N) para update e get.
 * Memoria: O(4*N)
 * Requisitos:
 * - TAG deve ter: compose(TAG), apply(T&) e construtor identidade.
 * - T e o tipo do dado nas folhas.
 * Exemplo de uso: DualSegTree<ll, MyTag> st(v);
 */

/* --- Exemplo de TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {

```

```

    ll mul = 1, add = 0;
    void compose(const Tag& t) {
        mul = (mul * t.mul) % MOD;
        add = (add * t.mul + t.add) % MOD;
    }
    void apply(ll& val) { val = (val * mul + add) % MOD; }
};
*/

```

```

2E5 template<typename T, typename TAG>
9C1 struct DualSegTree {
060     int N;
71A     vector<TAG> lazy;
E4B     vector<T> leaves;

11D     DualSegTree(int n) : N(n), lazy(4 * n), leaves(n) {}
5F6     DualSegTree(const vector<T>& v)
62C         : N(v.size()), lazy(4 * v.size()), leaves(v) {}

F08     void push(int no, int l, int r) {
F7F         int e = no << 1;
EE1         int d = e | 1;
115         lazy[e].compose(lazy[no]);
7CC         lazy[d].compose(lazy[no]);
47F         lazy[no] = TAG();
48C     }

130     void update(int no, int l, int r, int a, int b, const TAG& v) {
2E6         if (b < l || r < a) return;
02B         if (a <= l && r <= b) {
92C             lazy[no].compose(v);
505             return;
CA0         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
9AB         update(no << 1, l, m, a, b, v);
087         update((no << 1) | 1, m + 1, r, a, b, v);
667     }

629     T get(int no, int l, int r, int idx) {
893         if (l == r) {
4C5             T res = leaves[idx];
2E3             lazy[no].apply(res);
B50             return res;
BE7         }
FF9         push(no, l, r);
27E         int m = (l + r) >> 1;
52E         if (idx <= m) return get(no << 1, l, m, idx);
483         else return get((no << 1) | 1, m + 1, r, idx);
A24     }

A21     void update(int l, int r, const TAG& t) { update(1, 0, N - 1, l, r, t); }
AE4     T get(int idx) { return get(1, 0, N - 1, idx); }
6C2 };

```

1.6 Dynamic Segment Tree

```

/* Dynamic Segment Tree with Lazy Propagation (Range Update, Range Query)
 * Utilizada para intervalos gigantes (ex: 0 a 10^18).
 * Complexidade: O(Log N) por operacao.
 * Memoria: O(Q log N) onde Q e o numero de operacoes.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&),
 *   apply(const TAG&, ll, ll) e construtor identidade.
 * - TAG deve ter: apply(const TAG&), campo 'has' (bool) e
 *   construtor identidade.
 */

```

```

/* --- Exemplo de NODE e TAG ---
struct Tag {
    ll add = 0;
    bool has = false;

```

```

    void apply(const Tag& t) {
        add += t.add;
        has = true;
    }
};

struct Node {
    ll val = 0;
    static Node merge(const Node& l, const Node& r) {
        return {l.val + r.val};
    }
    void apply(const Tag& t, ll l, ll r) {
        val += t.add * (r - l + 1);
    }
};
*/

708 template<typename NODE, typename TAG>
770 struct DynamicSegTree {
126     struct InternalNode {
4EB         NODE node;
657         TAG tag;
74F         int l, r;
FB6         InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}

A61         void apply(const TAG& t, ll l, ll r) {
BE6             node.apply(t, l, r);
D21             tag.apply(t);
9C3         }
2FB     };

9B3     ll L, R;
4C1     vector<InternalNode> seg;

5C7     DynamicSegTree(ll l, ll r) : L(l), R(r) {
23E         seg.emplace_back();
76C     }

4F9     void push(int v, ll l, ll r) {
A16         if (!seg[v].tag.has) return;
568         if (l == r) return seg[v].tag = TAG();

579         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
203         if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }

769         ll mid = l + (r - l) / 2;
F3E         seg[seg[v].l].apply(seg[v].tag, l, mid);
0B7         seg[seg[v].r].apply(seg[v].tag, mid+1, r);

DD0         seg[v].tag = TAG();
AD7     }

352     void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) {
151         if (ql <= l && r <= qr) return seg[v].apply(t, l, r);

A48         push(v, l, r);
769         ll mid = l + (r - l) / 2;
441         if (ql <= mid) {
579             if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
2DD             update(seg[v].l, l, mid, ql, qr, t);
0B1         }
64D         if (qr > mid) {
203             if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
768             update(seg[v].r, mid + 1, r, ql, qr, t);
216         }

C5A         NODE left = (seg[v].l != -1) ? seg[seg[v].l].node : NODE();
C56         NODE right = (seg[v].r != -1) ? seg[seg[v].r].node : NODE();
799         seg[v].node = NODE::merge(left, right);
15E     }

EE0     NODE query(int v, ll l, ll r, ll ql, ll qr) {
5CF         if (v == -1 || ql > r || qr < l) return NODE();
3C9         if (ql <= l && r <= qr) return seg[v].node;

```

```

A48     push(v, l, r);
769     ll mid = l + (r - l) / 2;
211     return NODE::merge(query(seg[v].l, l, mid, ql, qr),
0CC         query(seg[v].r, mid + 1, r, ql, qr));
933 }

4CA void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
323 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
2A3 };

```

1.7 Lazy Segment Tree

```

/* Lazy Segment Tree (Range Update, Range Query)
 * Realiza atualizacoes e consultas em range.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(TAG, L, R) e
 *   construtor identidade.
 * - TAG deve ter: compose(TAG) e construtor identidade.
 */

/* --- Exemplo de NODE e TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void inline compose(const Tag& t) {
        add = (add * t.mul + t.add) % MOD;
        mul = (mul * t.mul) % MOD;
    }
};

struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node((l.val + r.val) % MOD);
    }
    void inline apply(const Tag& t, int l, int r) {
        val = (val * t.mul + t.add * (r - l + 1)) % MOD;
    }
};
*/

```

```

708 template<typename NODE, typename TAG>
2BA struct LazySegmentTree {
060     int N;
B1C     vector<NODE> seg;
71A     vector<TAG> lazy;

F0F     LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}

CCA     LazySegmentTree(const vector<int>& v)
9EB         : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
231         build(1, 0, N - 1, v);
5EE     }

63E void build(int no, int l, int r, const vector<int>& v) {
893     if (l == r) {
C3E         seg[no] = NODE(v[l]);
505         return;
B2E     }
27E     int m = (l + r) >> 1;
35E     build(no << 1, l, m, v);
C55     build(no << 1 | 1, m + 1, r, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[no << 1 | 1]);
A6F }

F08 void push(int no, int l, int r) {
27E     int m = (l + r) >> 1;
1EA     int e = no << 1, d = e | 1;

25F     seg[e].apply(lazy[no], l, m);

```

```

115     lazy[e].compose(lazy[no]);

4E1     seg[d].apply(lazy[no], m + 1, r);
7CC     lazy[d].compose(lazy[no]);

47F     lazy[no] = TAG();
6A3 }

130 void update(int no, int l, int r, int a, int b, const TAG& v) {
2E6     if (b < l || r < a) return;
02B     if (a <= l && r <= b) {
8C7         seg[no].apply(v, l, r);
92C         lazy[no].compose(v);
505         return;
81F     }
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
9AB     update(no << 1, l, m, a, b, v);
087     update((no << 1) | 1, m + 1, r, a, b, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
A1A }

38D NODE query(int no, int l, int r, int a, int b) {
2E6     if (b < l || r < a) return NODE();
83F     if (a <= l && r <= b) return seg[no];
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
AFE     return NODE::merge(query(no << 1, l, m, a, b),
074         query((no << 1) | 1, m + 1, r, a, b));
80A }

0D7 void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
36F NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
B0F };

```

1.8 Persistent Segment Tree

```

/* Persistent Segment Tree (Point Update, Range Query)
 * Permite consultas em versoes anteriores da arvore.
 * Complexidade: O(log N) por update e query.
 * Memoria: O(Q log N) onde Q e o numero de updates.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e
 *   construtor identidade.
 */

```

```

/* --- Exemplo de NODE (Soma) ---
struct Node {
    int val;
    Node(int v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

8A0 template<typename NODE>
68A struct PersistentSegmentTree {
126     struct InternalNode {
1DF         NODE data;
74F         int l, r;
E61         InternalNode() : data(NODE()), l(0), r(0) {}
B12     };

060     int N;
788     vector<InternalNode> st;
34F     vector<int> roots;

BC9     PersistentSegmentTree(int n) : N(n) {
F96         st.emplace_back(); // No 0 e o no nulo
ECB         roots.push_back(build(0, N - 1));
D5E     }

```

```

36C int build(int L, int R) {
4C0     int node = st.size();
F96     st.emplace_back();
1F6     if (L == R) return node;
57D     int mid = (L + R) / 2;
4FE     st[node].l = build(L, mid);
000     st[node].r = build(mid + 1, R);
777     st[node].data = NODE::merge(st[st[node].l].data, st[st[node].r].data);
192     return node;
55E }

C93 int update(int prev_root, int L, int R, int idx, NODE v) {
4C0     int node = st.size();
98F     st.push_back(st[prev_root]);
651     if (L == R) {
DC5         st[node].data = NODE::merge(st[node].data, v);
192         return node;
BE8     }
57D     int mid = (L + R) / 2;
82E     if (idx <= mid) st[node].l = update(st[prev_root].l, L, mid, idx, v);
102     else st[node].r = update(st[prev_root].r, mid + 1, R, idx, v);

777     st[node].data = NODE::merge(st[st[node].l].data, st[st[node].r].data);
192     return node;
B69 }

082 NODE query(int node, int L, int R, int i, int j) {
5EF     if (i > R || j < L) return NODE();
692     if (i <= L && R <= j) return st[node].data;
57D     int mid = (L + R) / 2;
B1E     return NODE::merge(query(st[node].l, L, mid, i, j),
521         query(st[node].r, mid + 1, R, i, j));
10E }

D6B void update(int idx, NODE v) {
69E     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
1BF }

3A5 NODE query(int version, int l, int r) {
191     return query(roots[version], 0, N - 1, l, r);
247 }
B9E };

```

1.9 Segment Tree

```

/* Segment Tree (Point Update, Range Query)
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(V), e construtor identidade.
 */

/* --- Exemplo de NODE (Soma com Update de Substituicao) ---
struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}

    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }

    // Para Point Set: val = v;
    // Para Point Add: val += v;
    inline void apply(int v) {
        val = v;
    }
};
*/

8A0 template<typename NODE>
383 struct SegTree {

```

```

060 int N;
01C vector<NODE> seg;

E7A SegTree(int n) : N(n), seg(4 * n) {}

3AD SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
231 build(1, 0, N - 1, v);
713 }

63E void build(int no, int l, int r, const vector<int>& v) {
893 if (l == r) {
C3E seg[no] = NODE(v[l]);
505 return;
B2E }
27E int m = (l + r) >> 1;
35E build(no << 1, l, m, v);
C55 build(no << 1 | 1, m + 1, r, v);
DEB seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
A6F }

663 void update(int no, int l, int r, int idx, int val) {
893 if (l == r) {
D88 seg[no].apply(val);
505 return;
EC9 }
27E int m = (l + r) >> 1;
B73 if (idx <= m) update(no << 1, l, m, idx, val);
99C else update(no << 1 | 1, m + 1, r, idx, val);
DEB seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
358 }

38D NODE query(int no, int l, int r, int a, int b) {
2E6 if (b < l || r < a) return NODE();
83F if (a <= l && r <= b) return seg[no];
27E int m = (l + r) >> 1;
AFE return NODE::merge(query(no << 1, l, m, a, b),
074 query(no << 1 | 1, m + 1, r, a, b));
365 }

9B0 void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
36F NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
81A };

```

1.10 Disjoint Sparse Table

```

/* Disjoint Sparse Table (Static Range Query)
* Realiza consultas em range em O(1) para qualquer operacao associativa.
* Complexidade: O(N log N) no build, O(1) na query.
* Memoria: O(N log N)
* Requisitos:
* - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
* - Operacao deve ser ASSOCIATIVA: f(f(a,b),c) = f(a,f(b,c)).
*/

```

```

/* --- Exemplo de NODE (Soma em Range) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

```

```

8A0 template<typename NODE>
906 struct DisjointSparseTable {
5F0 int N, K;
6F5 vector<vector<NODE>> st;

67A template<typename T>
68E DisjointSparseTable(const vector<T>& v) : N(v.size()) {
3AA if (N == 0) return;

```

```

475 K = __lg(2 * N - 1);
C78 st.assign(K, vector<NODE>(1 << K));

DDD for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);

C54 for (int j = 1; j < K; j++) {
1C0 int len = 1 << j;
C2C for (int m = len; m <= (1 << K); m += 2 * len) {
A34 if (m < N) st[j][m] = st[0][m];
6D6 for (int i = m + 1; i < min(N, m + len); i++)
0EA st[j][i] = NODE::merge(st[j][i - 1], st[0][i]);

C2E if (m - 1 < N) st[j][m - 1] = st[0][m - 1];
226 for (int i = m - 2; i >= m - len; i--)
604 st[j][i] = NODE::merge(st[0][i], st[j][i + 1]);
643 }
3F0 }
4D7 }

1F9 inline NODE query(int l, int r) {
434 if (l == r) return st[0][l];
B89 int j = __lg(l ^ r);
894 return NODE::merge(st[j][l], st[j][r]);
DB5 }
4D6 };

```

1.11 Sparse Table

```

/* Sparse Table (Static RMQ)
* Realiza consultas em range em tempo constante.
* Complexidade: O(N log N) no build, O(1) na query.
* Memoria: O(N log N)
* Requisitos:
* - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
* - A operacao DEVE ser idempotente: f(a, a) = a.
*/

```

```

/* --- Exemplo de NODE (Indice do Minimo) ---
struct Node {
    int val, pos;
    Node(int v = 2e9, int p = -1) : val(v), pos(p) {}

```

```

    static inline Node merge(const Node& l, const Node& r) {
        if (l.val <= r.val) return l;
        return r;
    }
};
*/

```

```

8A0 template<typename NODE>
7E9 struct SparseTable {
5F0 int N, K;
6F5 vector<vector<NODE>> st;
4A7 vector<int> lg;

67A template<typename T>
E4A SparseTable(const vector<T>& v) : N(v.size()) {
A77 K = (N > 0) ? __lg(N) : 0;
3A9 st.assign(K + 1, vector<NODE>(N));
lg.assign(N + 1, 0);
641 for (int i = 2; i <= N; i++) lg[i] = lg[i / 2] + 1;
BB5 for (int i = 0; i < N; i++) st[0][i] = NODE(v[i], i);
D3B for (int j = 1; j <= K; j++)
2CE for (int i = 0; i + (1 << j) <= N; i++)
285 st[j][i] = NODE::merge(st[j - 1][i],
4E1 st[j - 1][i + (1 << (j - 1))]);
A64 }
F93 }

762 NODE query(int l, int r) {
2C3 int j = lg[r - l + 1];
8F2 return NODE::merge(st[j][l], st[j][r - (1 << j) + 1]);
843 }

```

```
A18 };
```

2 Graphs

2.1 Lowest Common Ancestor (LCA)

```
11B #include "../data-structures/sparse-table/sparse-table.hpp"
```

```

/* LCA (Lowest Common Ancestor) via RMQ
* Encontra o ancestral comum mais proximo de dois vertices em uma arvore.
* Complexidade: O(N log N) no build, O(1) por query.
* Memoria: O(N log N)
* Requisitos:
* - Grafo deve ser uma arvore (conexo e aciclico).
* - Dependencia: sparse-table/sparse-table.hpp
* Metodos:
* - lca(u, v): Retorna o LCA de u e v.
* - dist(u, v): Retorna a distancia entre u e v.
* - is_ancestor(u, v): Verifica se u e ancestral de v.
* - parent(u): Retorna o pai de u.
*/

```

```

542 struct LCANode {
43D int dep, id;
89D LCANode(int d = 1e9, int pos = -1) : dep(d), id(pos) {}

5CF static inline LCANode merge(const LCANode& l, const LCANode& r) {
DC4 return l.dep < r.dep ? l : r;
505 }
DAD };

```

```

33E struct LCA {
060 int N;
41B vector<int> first, tour, d, p;
73B SparseTable<LCANode> st;

DCD LCA(int n, int root, const vector<vector<int>>& adj)
D75 : N(n), first(n), d(n), p(n, -1), st(vector<LCANode>()) {}

```

```

91C vector<int> tour_depths;
CBA tour.reserve(2 * n);
02D tour_depths.reserve(2 * n);

```

```

A12 auto dfs = [&](auto self, int u, int parent_id, int dep) -> void {
73F p[u] = parent_id;
701 d[u] = dep;
D86 first[u] = tour.size();
FD8 tour.push_back(u);
09A tour_depths.push_back(dep);
372 for (int v : adj[u]) {
F21 if (v == parent_id) continue;
207 self(self, v, u, dep + 1);
FD8 tour.push_back(u);
09A tour_depths.push_back(dep);
FF7 }
0D5 };

7D9 dfs(dfs, root, -1, 0);
9F1 st = SparseTable<LCANode>(tour_depths);
BCA }

```

```

789 int parent(int u) {
F60 return p[u];
47F }

```

```

310 int lca(int u, int v) {
B63 int l = first[u], r = first[v];
A13 if (l > r) swap(l, r);
369 return tour[st.query(l, r).id];
1E6 }

```

```

C11  int dist(int u, int v) {
05C      return d[u] + d[v] - 2 * d[lca(u, v)];
465  }

F31  bool is_ancestor(int u, int v) {
645      return lca(u, v) == u;
C22  }
39E  };

```

2.2 Hld Commutative Lazy

```
2F0 #include "../data-structures/segment-tree/lazy-segment-tree.hpp"
```

```

/* HLD Comutativo com Lazy Propagation
 * Para operacoes onde A + B = B + A.
 * Permite updates e queries em caminhos e subarvores.
 * Requisito: data-structures/segment-tree/lazy-segment-tree.hpp
 */

```

```

708 template<typename NODE, typename TAG>
403 struct HLD {
577     int n, t;
523     vector<int> p, sz, d, head, pos;
FFF     LazySegmentTree<NODE, TAG> st;

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833         : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {}

```

```

898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;

```

```

94B     auto dfs_sz = [&](auto self, int u) -> void {
267         sz[u] = 1;
839         int best_v = -1, max_sz = -1;
A8C         for (auto &v : g[u]) {
567             if (v == p[u]) continue;
ABE             d[v] = d[u] + 1; p[v] = u;
033             self(self, v);
CC3             sz[u] += sz[v];
D9D             if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
A7E         }
2D5         if (best_v != -1) {
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
F10                 if (g[u][i] == best_v && i != 0) {
D76                     swap(g[u][0], g[u][i]);
C2B                     break;
27A                 }
CAE             }
07D         }
A11     };

```

```

B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8         pos[u] = t++;
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
06E             int v = g[u][i];
567             if (v == p[u]) continue;
BD7             head[v] = (i == 0 ? head[u] : v);
033             self(self, v);
D08         }
113     };

```

```

A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);

```

```

759     vector<NODE> base(n);
830     for(int i = 0; i < n; i++)
101         base[pos[i]] = vals[i];
9F8     st = LazySegmentTree<NODE, TAG>(base);
188 }

```

```

D7C void update_path(int u, int v, const TAG& tag) {
0E0     while (head[u] != head[v]) {
44B         if (d[head[u]] > d[head[v]]) swap(u, v);
EC8         st.update(pos[head[v]], pos[v], tag);
025         v = p[head[v]];
DD0     }
0E0     if (d[u] > d[v]) swap(u, v);
07A     st.update(pos[u], pos[v], tag);
656 }

```

```

0F8 NODE query_path(int u, int v) {
ADA     NODE res;
0E0     while (head[u] != head[v]) {
44B         if (d[head[u]] > d[head[v]]) swap(u, v);
A0D         res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
025         v = p[head[v]];
41F     }
0E0     if (d[u] > d[v]) swap(u, v);
DC7     return NODE::merge(res, st.query(pos[u], pos[v]));
81B }

```

```

F81 void update_subtree(int u, const TAG& tag) {
675     st.update(pos[u], pos[u] + sz[u] - 1, tag);
20D }

```

```

51A NODE query_subtree(int u) {
0CF     return st.query(pos[u], pos[u] + sz[u] - 1);
747 }
AA5 };

```

2.3 Hld Commutative

```
6A0 #include "../data-structures/segment-tree/segment-tree.hpp"
```

```

/* HLD Comutativo (Update Pontual)
 * Para operacoes onde A + B = B + A.
 * Requisito: data-structures/segment-tree/segment-tree.hpp
 */

```

```

8A0 template<typename NODE>
403 struct HLD {
577     int n, t;
523     vector<int> p, sz, d, head, pos;
115     SegTree<NODE> st;

```

```

7EC HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {}

```

```

898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;

```

```

94B     auto dfs_sz = [&](auto self, int u) -> void {
267         sz[u] = 1;
839         int best_v = -1, max_sz = -1;
A8C         for (auto &v : g[u]) {
567             if (v == p[u]) continue;
ABE             d[v] = d[u] + 1; p[v] = u;
033             self(self, v);
CC3             sz[u] += sz[v];
1F6             if (sz[v] > max_sz) {
F66                 max_sz = sz[v];
DD0                 best_v = v;
D9D             }
A7E         }
2D5         if (best_v != -1) {
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
F10                 if (g[u][i] == best_v && i != 0) {
D76                     swap(g[u][0], g[u][i]);
C2B                     break;
27A                 }
CAE             }
07D         }

```

```

A11     };

B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8         pos[u] = t++;
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
06E             int v = g[u][i];
567             if (v == p[u]) continue;
BD7             head[v] = (i == 0 ? head[u] : v);
033             self(self, v);
D08         }
113     };

```

```

A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);

```

```

759     vector<NODE> base(n);
830     for(int i = 0; i < n; i++)
101         base[pos[i]] = vals[i];
E4E     st = SegTree<NODE>(base);
DE8 }

```

```

E1B void update(int u, const NODE& val) {
C7A     st.update(pos[u], val);
080 }

```

```

0F8 NODE query_path(int u, int v) {
ADA     NODE res;
0E0     while (head[u] != head[v]) {
44B         if (d[head[u]] > d[head[v]]) swap(u, v);
A0D         res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
025         v = p[head[v]];
41F     }
0E0     if (d[u] > d[v]) swap(u, v);
DC7     return NODE::merge(res, st.query(pos[u], pos[v]));
81B }

```

```

51A NODE query_subtree(int u) {
0CF     return st.query(pos[u], pos[u] + sz[u] - 1);
747 }
058 };

```

2.4 Hld Non Commutative Lazy

```
2F0 #include "../data-structures/segment-tree/lazy-segment-tree.hpp"
```

```

/* HLD Nao-Comutativo com Lazy Propagation
 * Para operacoes onde A + B != B + A (ex: Matrizas, Funcoes Lineares).
 * Requisito: DoubleNode que guarda versoes 'normal' e 'invertida' do merge.
 */

```

```

8A0 template<typename NODE>
0FF struct DoubleNode {
A30     NODE down, up;

F17     DoubleNode() : down(NODE()), up(NODE()) {}
77F     DoubleNode(const NODE& n) : down(n), up(n) {}
31A     DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

CFE     static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
2D1         return {
15C             NODE::merge(l.down, r.down),
F17             NODE::merge(r.up, l.up)
5D3         };
809     };

```

```

BBF template<typename TAG>
3CC void apply(const TAG& t, int l, int r) {
B08     down.apply(t, l, r);
2BF     up.apply(t, l, r);
8FB }
B7B };

```



```

708 template<typename NODE, typename TAG>
403 struct HLD {
577     int n, t;
523     vector<int> p, sz, d, head, pos;
C94     LazySegmentTree<DoubleNode<NODE>, TAG> st;

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
833     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

898         vector<vector<int>> g = adj;
227         d[root] = 0, p[root] = root;

94B         auto dfs_sz = [&](auto self, int u) -> void {
267             sz[u] = 1;
839             int best_v = -1, max_sz = -1;
A8C             for (auto &v : g[u]) {
567                 if (v == p[u]) continue;
ABE                 d[v] = d[u] + 1; p[v] = u;
033                 self(self, v);
CC3                 sz[u] += sz[v];
D9D                 if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
A7E             }
2D5             if (best_v != -1) {
BB6                 for (int i = 0; i < (int)g[u].size(); i++) {
F10                     if (g[u][i] == best_v && i != 0) {
D76                         swap(g[u][0], g[u][i]);
C2B                         break;
27A                     }
CAE                 }
07D             }
A11         };

89B         auto dfs_hld = [&](auto self, int u) -> void {
4C8             pos[u] = t++;
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
06E                 int v = g[u][i];
567                 if (v == p[u]) continue;
BD7                 head[v] = (i == 0 ? head[u] : v);
033                 self(self, v);
D08             }
113         };

A8A         dfs_sz(dfs_sz, root);
C07         head[root] = root;
C37         dfs_hld(dfs_hld, root);

9FC         vector<DoubleNode<NODE>> base(n);
B89         for(int i = 0; i < n; i++) base[pos[i]] = DoubleNode<NODE>(vals[i]);
54E         st = LazySegmentTree<DoubleNode<NODE>, TAG>(base);
B60     }

```

```

D7C void update_path(int u, int v, const TAG& tag) {
0E0     while (head[u] != head[v]) {
BF1         if (d[head[u]] > d[head[v]]) {
D76             st.update(pos[head[u]], pos[u], tag);
139             u = p[head[u]];
D79         } else {
EC8             st.update(pos[head[v]], pos[v], tag);
025             v = p[head[v]];
E8B         }
15B     }
B83     if (d[u] > d[v]) st.update(pos[v], pos[u], tag);
052     else st.update(pos[u], pos[v], tag);
855 }

```

```

0F8 NODE query_path(int u, int v) {
34B     NODE L, R;
0E0     while (head[u] != head[v]) {
BF1         if (d[head[u]] > d[head[v]]) {
0F3             L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
139             u = p[head[u]];
3B2         } else {
B5C             R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);

```

```

025         v = p[head[v]];
7D9     }
DBB     }
152     if (d[u] > d[v]) {
7E5         L = NODE::merge(L, st.query(pos[v], pos[u]).up);
D6C     } else {
806         R = NODE::merge(st.query(pos[u], pos[v]).down, R);
8A1     }
85E     return NODE::merge(L, R);
EB1 }

F81 void update_subtree(int u, const TAG& tag) {
675     st.update(pos[u], pos[u] + sz[u] - 1, tag);
20D }

51A NODE query_subtree(int u) {
DE6     return st.query(pos[u], pos[u] + sz[u] - 1).down;
588 }
068 };

```

2.5 Hld Non Commutative

```

6A0 #include "../data-structures/segment-tree/segment-tree.hpp"

```

```

/* HLD Nao-Comutativo (Update Pontual)
 * Para operacoes onde A * B != B * A.
 * Requisito: DoubleNode para gerenciar merges em ambas as direcoes.
 */

```

```

8A0 template<typename NODE>
0FF struct DoubleNode {
A30     NODE down, up;
F17     DoubleNode() : down(NODE()), up(NODE()) {}
77F     DoubleNode(const NODE& n) : down(n), up(n) {}
31A     DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

CFE     static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
2D1         return {
15C             NODE::merge(l.down, r.down),
F17             NODE::merge(r.up, l.up)
5D3         };
809     }
3DC };

```

```

8A0 template<typename NODE>
403 struct HLD {
577     int n, t;
256     vector<int> p, sz, d, head, pos, tour;
58B     SegTree<DoubleNode<NODE>> st;

```

```

7EC     HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
7F8     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), tour(n), st(n) {

```

```

898         vector<vector<int>> g = adj;
227         d[root] = 0, p[root] = root;

```

```

94B         auto dfs_sz = [&](auto self, int u) -> void {
267             sz[u] = 1;
839             int best_v = -1, max_sz = -1;
A8C             for (auto &v : g[u]) {
567                 if (v == p[u]) continue;
ABE                 d[v] = d[u] + 1; p[v] = u;
033                 self(self, v);
CC3                 sz[u] += sz[v];
D9D                 if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
A7E             }
2D5             if (best_v != -1) {
BB6                 for (int i = 0; i < (int)g[u].size(); i++) {
F10                     if (g[u][i] == best_v && i != 0) {
D76                         swap(g[u][0], g[u][i]);
C2B                         break;
27A                     }

```

```

CAE         }
07D     }
A11     };

B9B         auto dfs_hld = [&](auto self, int u) -> void {
4C8             pos[u] = t++;
6EA             tour[pos[u]] = u;
BB6             for (int i = 0; i < (int)g[u].size(); i++) {
06E                 int v = g[u][i];
567                 if (v == p[u]) continue;
BD7                 head[v] = (i == 0 ? head[u] : v);
033                 self(self, v);
D08             }
54A         };

```

```

A8A         dfs_sz(dfs_sz, root);
C07         head[root] = root;
C37         dfs_hld(dfs_hld, root);

```

```

9FC         vector<DoubleNode<NODE>> base(n);
830         for(int i = 0; i < n; i++)
9A8             base[pos[i]] = DoubleNode<NODE>(vals[i]);
19C         st = SegTree<DoubleNode<NODE>>(base);
997     }

```

```

E1B void update(int u, const NODE& val) {
353     st.update(pos[u], DoubleNode<NODE>(val));
443 }

```

```

0F8 NODE query_path(int u, int v) {
34B     NODE L, R;
0E0     while (head[u] != head[v]) {
BF1         if (d[head[u]] > d[head[v]]) {
0F3             L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
139             u = p[head[u]];
3B2         } else {
B5C             R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
025             v = p[head[v]];
7D9         }
DBB     }
6CE     if (d[u] > d[v]) L = NODE::merge(L, st.query(pos[v], pos[u]).up);
B9A     else R = NODE::merge(st.query(pos[u], pos[v]).down, R);

85E     return NODE::merge(L, R);
19F }

```

```

51A NODE query_subtree(int u) {
DE6     return st.query(pos[u], pos[u] + sz[u] - 1).down;
588 }
98B };

```

3 Extra

3.1 Hash Function

```

/* Lib Hash
 * Gera hash de trechos de codigo.
 * Ignora comentarios e espacos em branco.
 * O hash de uma linha com } e o hash de todo o codigo desde a { que a abriu.
 * Uso:
 * g++ hash.cpp -o hash
 * ./hash < code.cpp
 */

```

```

DE3 string getHash(string s){
E7F     ofstream("z.cpp") << s;
410     system("g++ -E -D -pD -fpreprocessed ./z.cpp | tr -d '[:space:]' | md5sum > sh");
F24     ifstream("sh") >> s;
A15     return s.substr(0, 3);
89F }

```

```
E8D int main(){
973 string l, t;
C3E stack<string> st({""});
C61 while(getline(cin, l)){
54F     t = l;
242     for(auto c : l)
A53         if(c == '{') st.push(""); else
733         if(c == '}') t = st.top()+l, st.pop();
189     cout << getHash(t) + " " + l << endl;
DB4     st.top() += t;
C14 }
692 }
```
